Architecture Alternatives and Engineering Trade-Offs

COMP-7431: Advanced Software Engineering | Week 4
CampusConnect architecture in action

Draw it. Assign it. Build it. Test it. Defend it.

Monday Morning: CampusConnect Needs a Shape

MONDAY, 9:00 A.M.

The team has requirements and an MVP. Now it must decide how CampusConnect will actually work.

THE TEAM'S FIRST ENGINEERING QUESTIONS

- What belongs inside CampusConnect—and what stays outside?
- Which component owns routing, content, and escalation?
- Where will approved content, logs, and secrets live?
- How will the browser call the backend?
- What happens when an external service fails?
- How will the team deploy, observe, and recover the system?

Architecture begins when these questions become named decisions, owned tasks, and testable behavior.

Week 4 Ends with Buildable Decisions

By the end of this lecture, students should be able to:

1. Draw the CampusConnect system boundary and external actors
2. Assign responsibilities to components and team members
3. Select storage, movement, and retention rules for each data type
4. Write concrete interface and dependency contracts
5. Implement security, deployment, recovery, and monitoring choices
6. Compare alternatives and record a defensible architecture decision

Architecture Becomes Visible through Actions

Each architecture idea produces something the team can inspect:

- Boundary → draw what CampusConnect owns and excludes
- Responsibility → name the component and student engineer who owns it
- Data → choose a store and define movement, access, and deletion
- Interface → write the request, response, errors, and timeout
- Security → mark every trust transition and add a control
- Operations → specify deployment, monitoring, rollback, and recovery

If the team cannot point to an artifact, an implementation, or a test, the decision is still only a slogan.

The Team Converts Requirements into Tasks

WHAT THE TEAM BRINGS INTO THE ROOM

- Five supported campus services
- Approved sources must be visible
- High-risk topics must escalate safely
- No student-record access in the MVP
- A small semester team and limited budget

WHAT THE TEAM MUST LEAVE WITH

- A system-context drawing
- Named components and owners
- A data and storage plan
- Interface and failure contracts
- A deployment and recovery plan
- An Architecture Decision Record

The meeting is successful when each major decision becomes a concrete task in the backlog.

Task 1: Draw the Boundary before the Boxes

The student engineer starts with a system-context view:

- Put CampusConnect in the center
- Add students, content editors, and service departments
- Add the model, search, email, or identity providers it may call
- Mark SIS records, payments, and case management outside the MVP
- Label each connection with the action or data crossing it
- Write an owner next to every external system and dependency

The boundary prevents the MVP from quietly absorbing work the team never agreed to own.

CampusConnect Has Three Practical Boundaries

INSIDE CAMPUSCONNECT

- Student web interface
- Routing and escalation rules
- Approved-content service
- Audit and outcome measures

CAMPUS PEOPLE AND SYSTEMS

- Students use the service
- Content owners approve guidance
- Departments receive escalations
- Identity may be added later

EXTERNAL PROVIDERS

- Optional language model
- Hosting and database service
- Email or notification service
- Monitoring service

Every crossing needs a contract, an owner, and a defined failure response.

Students Assign Work to Components and Owners

STUDENT EXPERIENCE

Frontend module: collect a need, show the source and next step, allow correction, meet keyboard and screen-reader expectations.

Owner: frontend engineer.

ROUTING AND CONTENT

Backend modules: validate input, apply policy, query approved content, return a safe result, and record the route.

Owners: backend and data engineers.

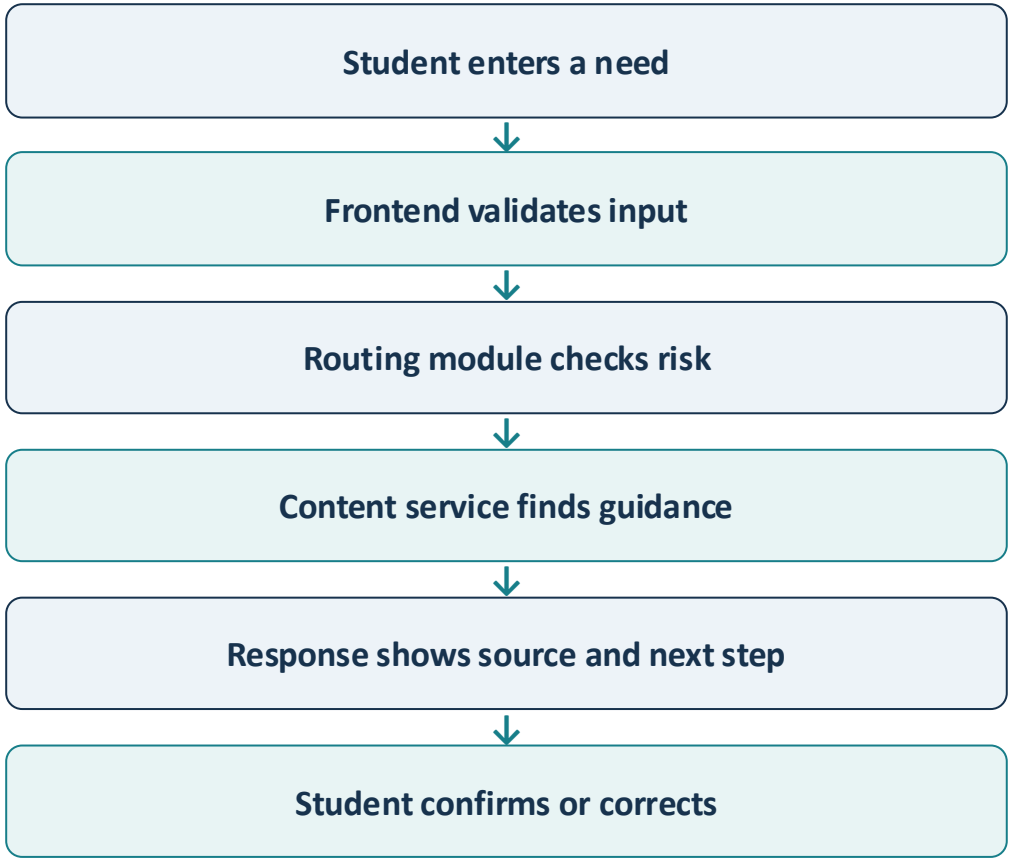
OPERATIONS AND GOVERNANCE

Content workflow, tests, deployment, secrets, logs, alerts, and rollback.

Owners: content steward and platform/security reviewer.

A component is useful when its responsibility, interface, owner, and failure behavior are all clear.

Follow One Student Request through the System



WHAT THE ENGINEERS IMPLEMENT

- A typed request and response
- Validation before routing
- A deterministic high-risk check
- A query against approved content
- Source and escalation fields
- A recorded outcome for later evaluation

Task 2: Inventory the Data before Choosing Storage

The team creates a simple data inventory:

- Name each data set: approved content, request, outcome, log, secret
- Identify its owner and authoritative source
- Mark it public, internal, sensitive, or prohibited
- State who may read, create, update, and delete it
- Choose the system of record—not merely a convenient folder
- Define movement, retention, backup, and disposal

Use synthetic data whenever real student data is unnecessary.

Different Data Needs Different Ownership

APPROVED SERVICE CONTENT

Owner: campus content steward.
Store: governed relational or document data.
Rule: version every change and preserve the source.

ROUTING EVENTS

Owner: CampusConnect team.
Store: minimized event record.
Rule: use a request ID; avoid names and unnecessary free text.

SECRETS AND CONFIGURATION

Owner: platform administrator.
Store: deployment secret manager.
Rule: never place keys in source code, slides, or shared notes.

The data plan states authority, purpose, access, retention, and deletion—not just a product name.

Google Drive, GitHub, and the Runtime Have Different Jobs

GOOGLE DRIVE

Use for stakeholder notes, approved source documents, meeting artifacts, and instructor-facing drafts.

Do not use it as the live application database.

GITHUB

Use for code, ADRs, schemas, infrastructure files, issues, tests, and synthetic fixtures.

Do not commit credentials or real student records.

RUNTIME SERVICES

Use a managed database for application data and a secret manager for API keys and connection strings.

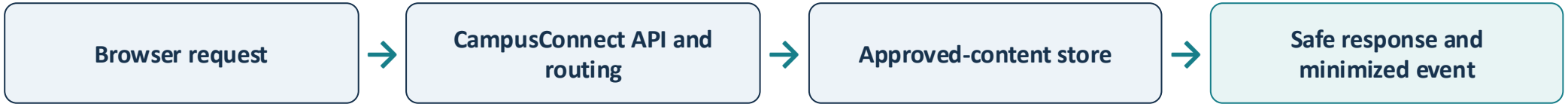
Back up and restrict access separately.

Convenience does not erase the security and lifecycle obligations of the data.

The Team Draws How Data Moves

CampusConnect moves only the data each step needs:

- Browser sends { need, locale } over HTTPS
- API validates before routing
- Policy receives no student identifier
- Content query returns approved records only
- Response includes source and next step
- Event keeps request ID, route, latency, and outcome—not unnecessary text



Retention Rules Become Scheduled Work

KEEP WHILE AUTHORITATIVE

- Approved service content
- Source and owner
- Version and review date
- Active escalation path

REVIEW OR DELETE

- Expired content
- Raw test sessions
- Temporary exports
- Old logs beyond policy

ENGINEERING EVIDENCE

- Retention field in inventory
- Scheduled cleanup job
- Deletion test
- Backup and restore record

“Retain for 30 days” is not complete until a job deletes the data and a test proves it.

Task 3: Turn Every Connection into a Contract

A line between boxes becomes engineering work when the team defines:

- Who initiates the interaction
- The request fields and validation rules
- The response fields and meaning
- Authentication and authorization needs
- Timeout, retry, and rate limits
- Error codes and safe fallback behavior
- Versioning and compatibility expectations

A labeled arrow should lead to an API specification, function signature, event schema, or workflow rule.

A CampusConnect API Contract Is Concrete

REQUEST FROM THE BROWSER

POST /api/route with { need, locale }

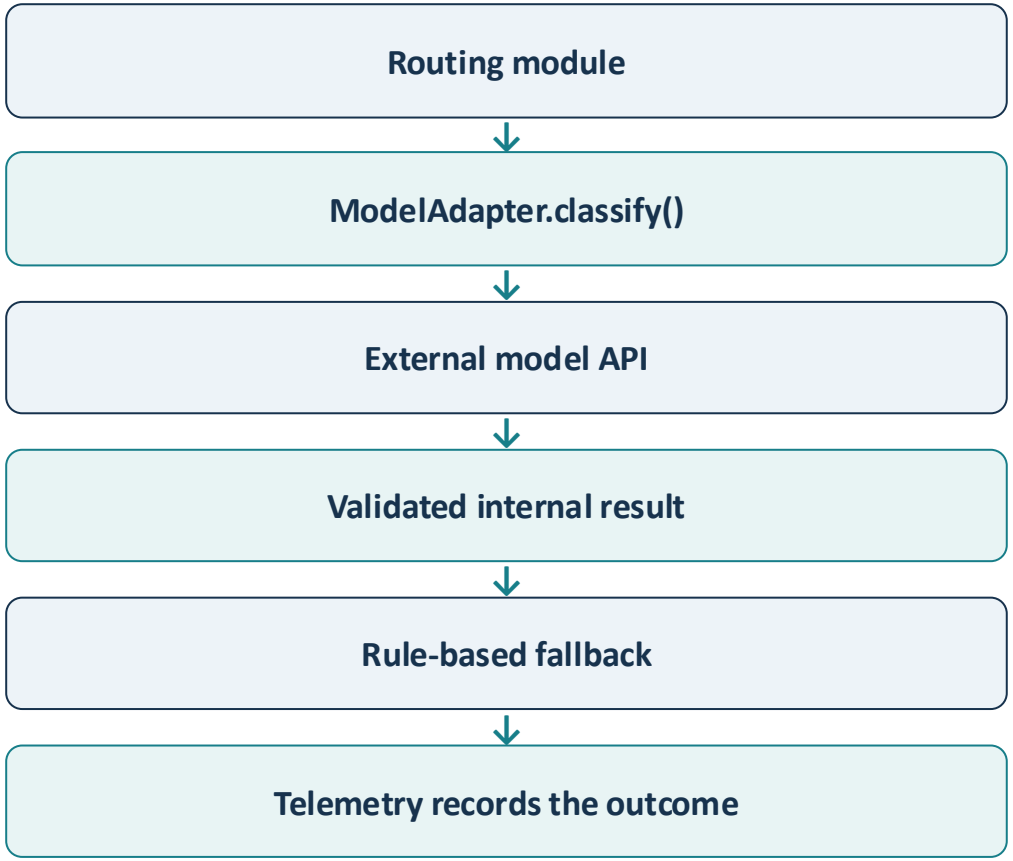
SUCCESS RESPONSE

{ service, next_step, source_url, escalation, request_id }

FAILURE CONTRACT

400 for invalid input; 503 with a safe human path when a required provider is unavailable.

External Services Sit behind an Adapter



WHAT THE STUDENT ENGINEER BUILDS

- One internal interface
- Provider-specific code inside the adapter
- A short timeout and bounded retry
- Input and output validation
- A fake adapter for tests
- A safe fallback when the provider fails

Communication Style Follows the User Journey

SYNCHRONOUS REQUEST

The student waits for a route. The browser calls the API and receives a result, correction path, or safe failure immediately.

ASYNCHRONOUS EVENT

The system records a minimized outcome after the response. Retry is allowed because the student is no longer waiting.

SCHEDULED WORK

A nightly job checks content review dates, removes expired temporary data, and refreshes the search index.

Students implement immediate work, background work, and scheduled work differently because their failure and timing expectations differ.

Task 4: Mark Every Trust Boundary

The team reviews each place where identity, privilege, or data changes:

- Browser input enters the application
- A public user reaches an administrative function
- Backend code reads or changes governed content
- A secret is used to call an external provider
- Logs leave the application for a monitoring service
- A content editor publishes guidance
- A high-risk request moves to a human service

At every boundary, students add validation, authorization, minimization, or a safe failure.

Identity and Access Become Implemented Rules

PUBLIC STUDENT PATH

Allow anonymous routing for the MVP. Do not request a login or student ID when the outcome does not require identity.

CONTENT-EDITOR PATH

Require campus sign-in, an editor role, and an approval step before a content change becomes visible.

SERVICE-TO-SERVICE PATH

Give the application only the database and provider permissions it needs. Store credentials in protected deployment configuration.

Authentication proves identity; authorization still decides what that identity may do.

Security and Privacy Change the Build

INPUT AND APPLICATION

Validate length and type, encode displayed content, rate-limit abuse, and keep policy rules outside prompts and page code.

DATA AND LOGS

Collect only what the MVP needs, avoid real student records, redact secrets, protect administrative changes, and expire temporary data.

DELIVERY AND REVIEW

Scan dependencies, protect the main branch, review pull requests, test authorization, and verify the deployed configuration.

Security is complete only when a control is implemented, tested, and owned.

A High-Risk Request Takes a Different Path

LOW-RISK PATH

"Where is the tutoring center?" CampusConnect returns an approved location, hours, source, and correction option.

HIGH-RISK PATH

"I may lose my financial aid today." A deterministic rule stops general generation and presents the approved financial-aid contact and urgent next step.

WHAT THE TEAM IMPLEMENTS

- A reviewed high-risk topic list
- A rule that runs before optional generation
- Approved escalation content
- A visible source and human contact
- A regression test for every high-risk route
- A log that records the route without exposing unnecessary text

Task 5: Package a Deployable Slice

The team now turns the component view into a running environment:

- Keep the MVP as one deployable web application
- Separate frontend, routing, content, audit, and adapter modules
- Use one governed managed database
- Configure secrets in the hosting environment
- Build and test through GitHub Actions
- Deploy first to a staging environment
- Add health checks, logs, rollback, and a support owner

A deployment diagram is useful only when the team can turn every box into configuration and every arrow into a verified connection.

CampusConnect Does Not Need Microservices Yet

Modular monolith for the MVP	Microservices from the start
One deployment and one release version	Several deployments and versions
Modules communicate in process	Services communicate across a network
One small team can review the whole change	Ownership and coordination must be explicit
Transactions and debugging stay simpler	Distributed failures and tracing appear immediately
Modules can be extracted later if evidence requires	Independent scaling is available—but adds operations

CampusConnect chooses a modular monolith because current team size and traffic do not justify distributed-system complexity.

Each Deployment Box Creates Student Work

CampusConnect uses one deployable application with separated modules:

- Repository: code, ADRs, tests, schemas, and deployment files
- CI pipeline: lint, unit tests, integration tests, dependency checks
- Staging: synthetic content and smoke tests before release
- Web host: runs the application and exposes a health endpoint
- Managed database: stores approved content and minimized events
- Secret store: supplies credentials without committing them
- Monitoring: receives structured logs, measures, and alerts

The deployment is not finished when the URL opens; it is finished when the team can verify, observe, and reverse it.

Scaling Starts with a Measured Bottleneck

The team does not “add scalability.” It responds to evidence:

- Slow content query → add an index and measure again
- Repeated approved reads → add a short cache with invalidation
- Model latency → shorten the timeout and preserve deterministic fallback
- Many concurrent requests → add application instances behind the host
- Expensive analytics → move it to asynchronous or scheduled work
- Growing content → paginate, index, and measure retrieval quality
- Rising cost → track calls and establish a budget alert

CampusConnect scales one constrained path at a time; premature distribution would create more failure modes than value.

The Team Rehearses Recovery before Demo Day

FAILURE: BAD RELEASE

- Health check fails
- Smoke test detects the issue
- Team rolls back to the prior image
- Owner records the incident

FAILURE: DATABASE CHANGE

- Migration is versioned
- Backup is verified
- Restore is rehearsed
- Data compatibility is checked

FAILURE: PROVIDER OUTAGE

- Timeout stops waiting
- Circuit or limit reduces repeated calls
- Deterministic fallback remains available
- Alert identifies the dependency

A recovery plan becomes credible only after the students execute it and preserve the evidence.

Observability Tells Students What to Do Next

CampusConnect signal	Student-team response
Routing latency rises	Trace API, policy, database, and provider time
Fallback use increases	Check provider health and recent code changes
Correction rate rises	Review routing cases and content gaps
Expired content is served	Fix review workflow and index refresh
Authorization failures spike	Investigate misuse or configuration change

Students instrument logs, measures, and request traces so a symptom leads to an owner and an investigation.

The Team Compares Real Alternatives

CampusConnect alternative	What the student engineers would build
Curated directory	Search page, approved records, filters, source links
Guided workflow	Clarifying questions, rules, correction, escalation
Search with retrieval	Index, ranking, citations, freshness checks
General AI chatbot	Prompts, model calls, guardrails, broad evaluation
Controlled hybrid	Rules for risk plus curated retrieval and optional AI
Build recommendation	Modular monolith with controlled hybrid routing
Why it wins now	Best fit for traceability, safe failure, and team capacity

The preferred architecture wins because its work and risks fit the evidence—not because it contains the newest technology.

Technology Choices Create Future Constraints

Technology choice	Immediate student action	Constraint created
Python FastAPI or Node API	Define modules and OpenAPI contract	Team must maintain that language and ecosystem
PostgreSQL / Supabase	Create schema, migrations, roles, backup	Relational model and provider features shape change
GitHub Actions	Automate tests and deployment	Workflow permissions and runner availability matter
Managed web host	Configure environments, health, and rollback	Platform limits, cost, and vendor features apply
External model API	Build adapter, timeout, validation, fallback	Latency, pricing, policy, and availability remain external
Hosted monitoring	Instrument structured telemetry	Data exposure, retention, and operating cost must be governed
Decision rule	Prefer replaceable tools behind stable contracts	Switching still costs time, migration, and retesting

Prototype the Decision That Could Change the Design

WEAK EXPERIMENT

"Build the complete CampusConnect chatbot and see whether students like it."

FOCUSED EXPERIMENT

"Compare guided rules, curated retrieval, and optional AI on 30 representative requests and five high-risk cases."

STUDENT ENGINEERING PACKAGE

- Name the decision the test will inform
- Use synthetic, representative requests
- Measure correct route, source visibility, latency, and fallback
- Include ambiguous and adversarial cases
- Record the failure threshold before testing
- Keep the prototype narrow and disposable
- Update the architecture decision with the result

ADR-004 Records the CampusConnect Decision

CONTEXT AND DECISION

1. Five-service MVP with mixed consequence
2. Approved sources and safe escalation required
3. Build one modular web application
4. Use deterministic routing for high-risk topics
5. Use curated retrieval for supported guidance
6. Isolate optional AI behind an adapter

CONSEQUENCES AND FOLLOW-UP

7. More testing than a simple directory
8. Model failure cannot disable safe routing
9. Content governance remains mandatory
10. Deploy through CI with protected secrets
11. Measure correction, fallback, latency, and cost
12. Revisit if AI adds little routing value

The ADR gives future students the reason, consequences, owner, confidence, and review trigger—not merely the chosen tools.

Good Architecture Makes the Next Action Clear

“Architecture is not a vocabulary test. It is a shared plan for building, protecting, delivering, and changing the system.”

- Draw what CampusConnect owns
- Assign every responsibility
- Put each data type in the right place
- Write contracts for every connection
- Mark trust boundaries and controls
- Deploy with rollback and recovery
- Instrument signals that lead to action
- Record trade-offs and review triggers

Next week: turn these decisions into detailed design, data models, interfaces, and working prototypes.